

✓ Premier apprentissage

1. Chargement du jeu de données

Jeux de caractères manuscrits proposé par scikitlearn Représentation des données

```
%matplotlib inline

from sklearn.datasets import load_digits
digits = load_digits()

# un index quelconque
idx = 24

# list des clefs du dictionnaire
keyList = list(digits.keys())
print("Clefs : ",keyList)

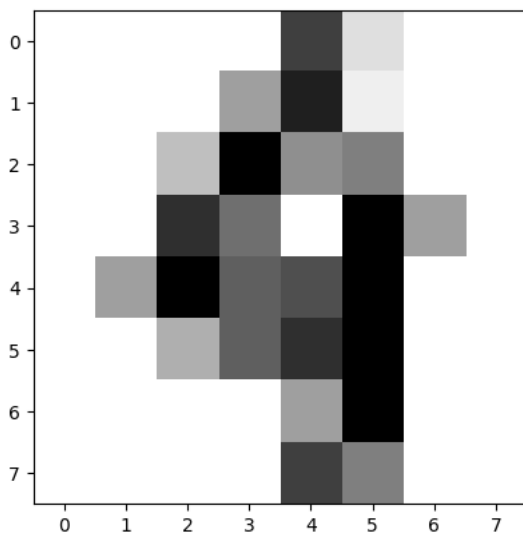
# représentation d'un élément dans data
print("Data : ", digits.data[idx])

# annotation d'un élément de data
print("Annotation :",digits.target[idx])

import matplotlib.pyplot as plt

plt.imshow(digits.images[idx], cmap='gray_r')
plt.show()
```

Clefs : ['data', 'target', 'frame', 'feature_names', 'target_names', 'images', 'DESCR']
 Data : [0. 0. 0. 0. 12. 2. 0. 0. 0. 0. 0. 6. 14. 1. 0. 0. 0. 0.
 4. 16. 7. 8. 0. 0. 0. 0. 13. 9. 0. 16. 6. 0. 0. 6. 16. 10.
 11. 16. 0. 0. 0. 0. 5. 10. 13. 16. 0. 0. 0. 0. 0. 6. 16.
 0. 0. 0. 0. 0. 0. 12. 8. 0. 0.]
 Annotation : 4



```
from sklearn.datasets import get_data_home
print(get_data_home())
```

```
/root/scikit_learn_data
```

✓ 2. Construction du jeu de données

Définition de la fonction `split_train_test`, son rôle est de mélanger le jeu de données et de le séparer en deux en fonction d'un ratio (par défaut 0.2 soit 20%). Le ratio exprime la partie réserver pour effectuer les calculs de performances du model.

Elle prend en paramètre d'entrée un tableau de vecteurs, où chaque vecteur représente une données et un tableau contenant les classes correspondantes à chaque vecteurs.

```
import numpy as np

def split_train_test( data, annotation, ratio=0.2, seed=42 ):
    np.random.seed(seed)
```

```

liste = list(zip(data, annotation))
np.random.shuffle(liste)
data, annotation = zip(*liste)
size_test = int( len(data) * (1-ratio) )
return np.array(data[:size_test]), np.array(annotation[:size_test]),\
        np.array(data[size_test+1:]), np.array(annotation[size_test+1:]))

trainX, trainY, testX, testY = split_train_test( digits.data, digits.target)

print(trainY)

print("Taille du jeu du jeu d'apprentissage {} et taille du jeux de test {}".format(trainX.shape, testX.shape))

# exemple du caractère à la position 24
idx = 0
print("Caractère à la position {} dans le jeu de données, il est annoté ({}).format( idx, trainY[idx]))
plt.imshow(trainX[idx].reshape(8,8), cmap='gray_r')
plt.show()

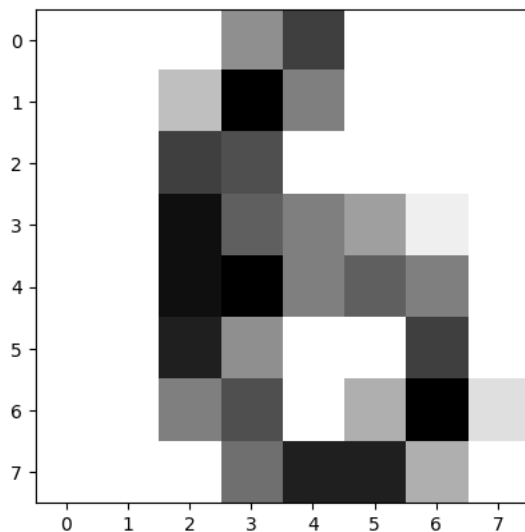
```

```

[6 9 3 ... 0 7 3]
Taille du jeu du jeu d'apprentissage (1437, 64) et taille du jeux de test (359, 64)

```

Caractère à la position 0 dans le jeu de données, il est annoté (6)



3. Préparation des données

Les images sont en nuance de gris sur 16 valeurs, les neurones de notre modèle sont de type on/off, on va donc devoir convertir nos jeux

Pour les annotations nous avons 10 classes la classe 0, classe 1 ... classe 9. Pour la même raison que les inputs on va devoir choisir une classe et redéfinir l'annotation à True pour la classe choisi et à False pour toutes les autres.

```

def modif_input( dataset, seuil = 4 ):
    for n in range(len(dataset)):
        for i in range(len(dataset[n])):
            dataset[n][i] = 1 if dataset[n][i] > seuil else 0
    return dataset

trainX = modif_input(trainX)
testX = modif_input(testX)

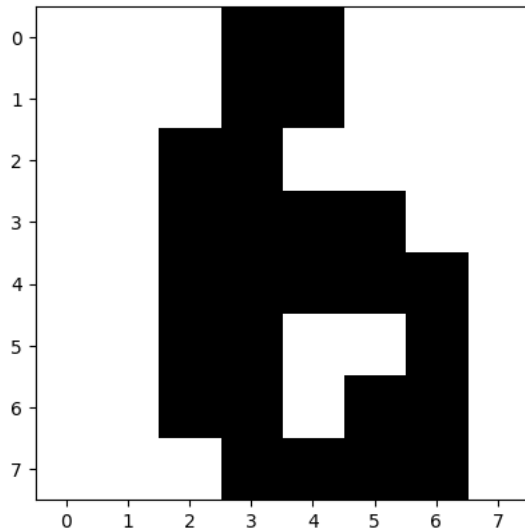
def modif_output( dataset, classe = 3 ):
    dataset = np.array(list(map( lambda x: x==classe, dataset )))
    return dataset

trainY = modif_output(trainY)
testY = modif_output(testY)

# exemple du caractère à la position 24
idx = 0
print("Caractère à la position {} dans le jeu de données, il est annoté ({}).format( idx, trainY[idx]))
plt.imshow(trainX[idx].reshape(8,8), cmap='gray_r')
plt.show()

```

Caractère à la position 0 dans le jeu de données, il est annoté (False)



4. Modèle d'apprentissage basé sur HEBB

Fonction fit pour l'apprentissage par renforcement des poids, apprentissage binaire, appartient à une classe ou n'appartient pas, retour le tableau des poids du model.

Fonction predict pour classer un tableau de vecteur donné.

```
import numpy as np

# apprentissage
def fit( X, Y ):
    weight = np.array([], float)
    # initialisation des poids
    for i in range(len(X[0])):
        weight = np.append( weight, 1 )

    # pour chaque vecteur du jeu de données
    for n in range(len(X)):
        for i in range(len(X[n])):
            weight[i] = weight[i] + X[n][i]*Y[n]

    return weight

# classification
def predict( X, weight, seuil = 2000 ):
    Y = []
    for n in range(len(X)):
        w = 0
        for i in range(len(X[n])):
            w = w + X[n][i]*weight[i]
        Y.append( w > seuil )
    return Y
```

5. Mesure de performance

Fonction calculant la précision et le rappel Fonction calculant la moyenne harmonique de la précision et du rappel

```
from collections import Counter

def performance( test, predic ):
    couple = list(zip(test,predic))
    nbrCouple = Counter(couple)
    TruePositif = nbrCouple[(True,True)]
    FalsePositif = nbrCouple[(False,True)]
    TrueNegatif = nbrCouple[(False,False)]
    FalseNegatif = nbrCouple[(True,False)]

    total_elements = TruePositif + FalsePositif + TrueNegatif + FalseNegatif

    preci = (TruePositif/(TruePositif+FalsePositif)) if ((TruePositif+FalsePositif)>0) else 0
    rappel = (TruePositif/(TruePositif+FalseNegatif)) if ((TruePositif+FalseNegatif)>0) else 0
    accuracy = (TruePositif + TrueNegatif) / total_elements if total_elements > 0 else 0
```

```
# Calcul du F1-score (moyenne harmonique de précision et rappel)
f1_score = (2 * preci * rappel) / (preci + rappel) if (preci + rappel) > 0 else 0

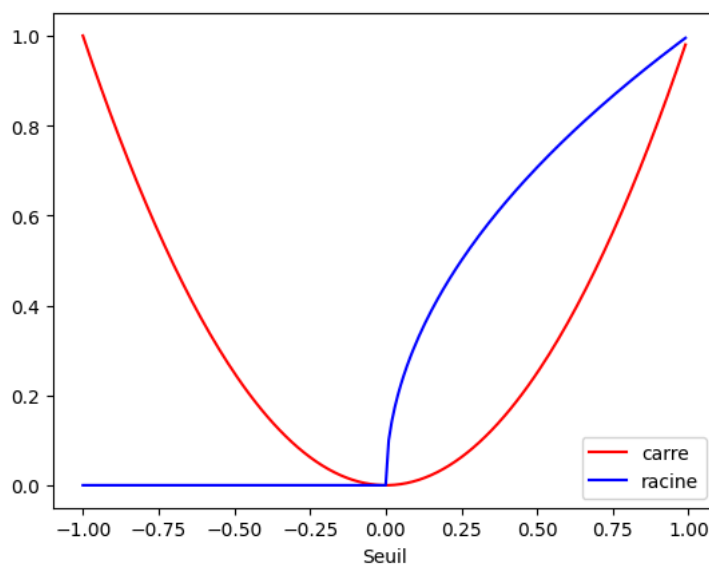
return preci, rappel, accuracy, f1_score
```

6. Exemple courbe

Code pour tracer un courbe

```
abscisse = [ i/100 for i in range(-100,100)]
courbe_un = []
courbe_deux = []
for x in abscisse:
    courbe_un.append( x**2 )
    courbe_deux.append( x**0.5 )

plt.plot(abscisse, courbe_un,"r-", label="carre")
plt.plot(abscisse, courbe_deux, "b-", label="racine")
plt.legend()
plt.xlabel("Seuil")
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

weights = fit(trainX, trainY)

seuils = np.arange(2200, 3400, 50)

sensibilites = []
precisions = []
accuracies = []
f1_scores = []
scores_f1 = []
scores_moyens = []

# Pour chaque seuil
for seuil in seuils:
    predictions = predict(testX, weights, seuil)
    precision, rappel, accuracy, f1_score_val = performance(testY, predictions)
    precisions.append(precision)
    sensibilites.append(rappel)
    f1_scores.append(f1_score_val)

    # Score F1 : moyenne harmonique
    if precision + rappel > 0:
        f1 = 2 * (precision * rappel) / (precision + rappel)
    else:
        f1 = 0
    scores_f1.append(f1)

    # Score moyen : moyenne arithmétique
    scores_moyens.append((precision + rappel) / 2)

plt.figure(figsize=(12, 7))
```

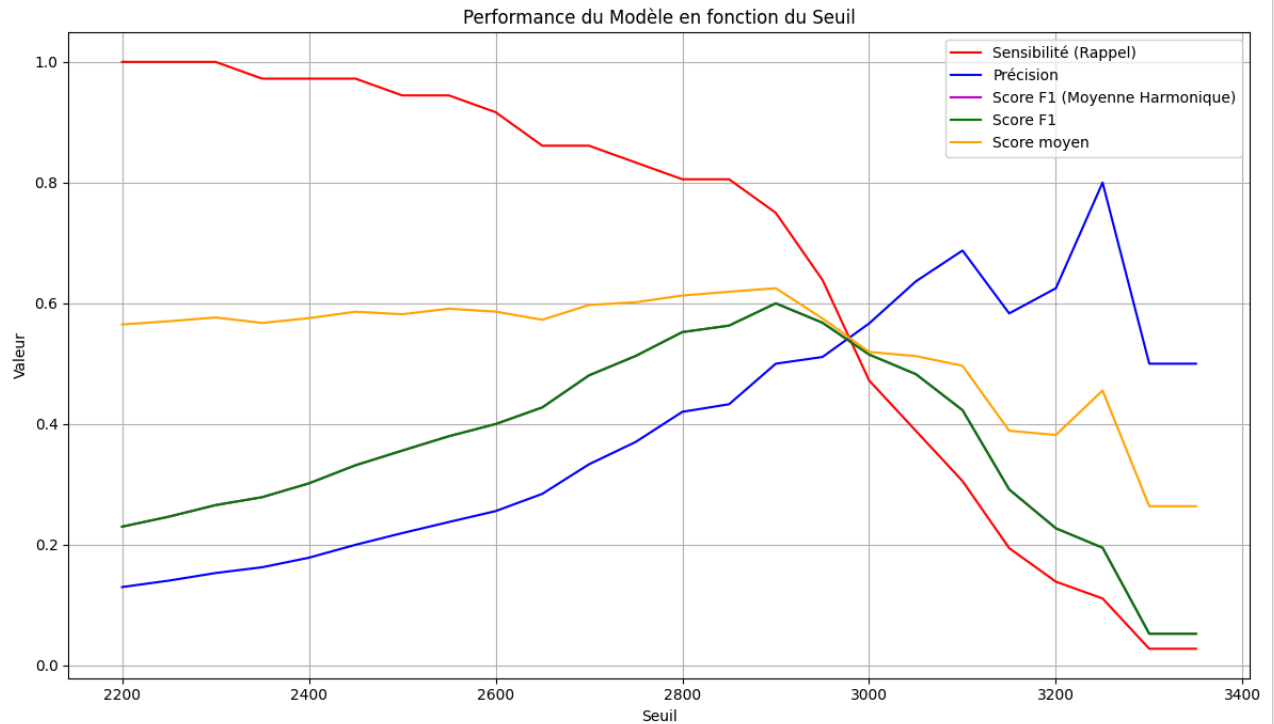
```

plt.plot(seuils, sensibilites, 'r-', label='Sensibilité (Rappel)')
plt.plot(seuils, precisions, 'b-', label='Précision')
plt.plot(seuils, f1_scores, 'm-', label='Score F1 (Moyenne Harmonique)')
plt.plot(seuils, scores_f1, 'g-', label='Score F1')
plt.plot(seuils, scores_moyens, 'orange', label='Score moyen')

plt.xlabel('Seuil')
plt.ylabel('Valeur')
plt.title('Performance du Modèle en fonction du Seuil')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

```



8. Courbe ROC

```

import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# Recalculate metrics for class 6 to include FPR
weights_class6 = fit(trainX, trainY)

seuils_roc = np.arange(2200, 3400, 10)

tpr_class6 = []
fpr_class6 = []

for seuil in seuils_roc:
    predictions_class6 = predict(testX, weights_class6, seuil)

    # Calculate TruePositif, FalsePositif, TrueNegatif, FalseNegatif
    couple = list(zip(testY, predictions_class6))
    nbrCouple = Counter(couple)

    TruePositif = nbrCouple[(True, True)]
    FalsePositif = nbrCouple[(False, True)]
    TrueNegatif = nbrCouple[(False, False)]
    FalseNegatif = nbrCouple[(True, False)]

    # True Positive Rate (rappel/sensitivity)
    tpr = (TruePositif / (TruePositif + FalseNegatif)) if ((TruePositif + FalseNegatif) > 0) else 0

```

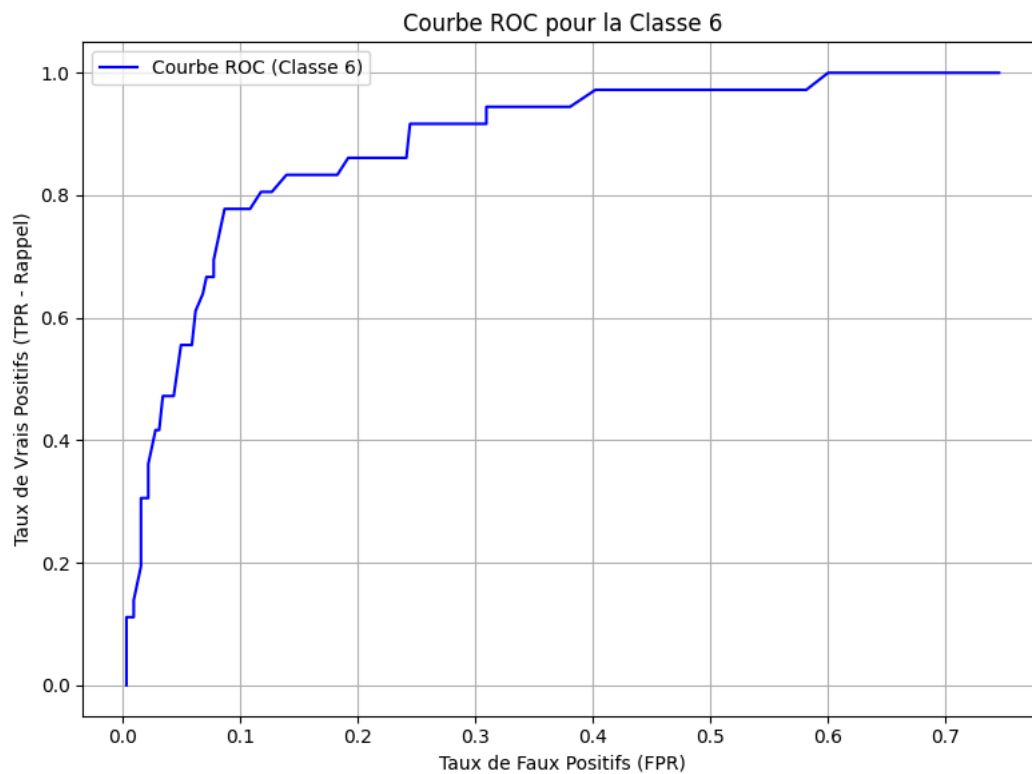
```

# False Positive Rate
fpr = (FalsePositif / (FalsePositif + TrueNegatif)) if ((FalsePositif + TrueNegatif) > 0) else 0

tpr_class6.append(tpr)
fpr_class6.append(fpr)

# Plot ROC curve for class 6
plt.figure(figsize=(8, 6))
plt.plot(fpr_class6, tpr_class6, 'b-', label='Courbe ROC (Classe 6)')
plt.xlabel('Taux de Faux Positifs (FPR)')
plt.ylabel('Taux de Vrais Positifs (TPR - Rappel)')
plt.title('Courbe ROC pour la Classe 6')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```



✓ Premier apprentissage

1. Chargement du jeu de données

Jeux de caractères manuscrits proposé par scikitlearn Représentation des données

```
%matplotlib inline

from sklearn.datasets import load_digits
digits = load_digits()

# un index quelconque
idx = 24

# list des clefs du dictionnaire
keyList = list(digits.keys())
print("Clefs : ",keyList)

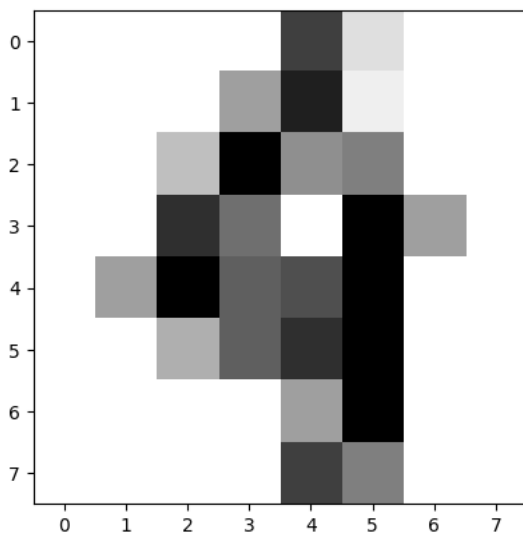
# représentation d'un élément dans data
print("Data : ", digits.data[idx])

# annotation d'un élément de data
print("Annotation :",digits.target[idx])

import matplotlib.pyplot as plt

plt.imshow(digits.images[idx], cmap='gray_r')
plt.show()
```

Clefs : ['data', 'target', 'frame', 'feature_names', 'target_names', 'images', 'DESCR']
 Data : [0. 0. 0. 0. 12. 2. 0. 0. 0. 0. 0. 6. 14. 1. 0. 0. 0. 0.
 4. 16. 7. 8. 0. 0. 0. 0. 13. 9. 0. 16. 6. 0. 0. 6. 16. 10.
 11. 16. 0. 0. 0. 0. 5. 10. 13. 16. 0. 0. 0. 0. 0. 6. 16.
 0. 0. 0. 0. 0. 0. 12. 8. 0. 0.]
 Annotation : 4



```
from sklearn.datasets import get_data_home
print(get_data_home())
```

```
/root/scikit_learn_data
```

✓ 2. Construction du jeu de données

Définition de la fonction `split_train_test`, son rôle est de mélanger le jeu de données et de le séparer en deux en fonction d'un ratio (par défaut 0.2 soit 20%). Le ratio exprime la partie réserver pour effectuer les calculs de performances du model.

Elle prend en paramètre d'entrée un tableau de vecteurs, où chaque vecteur représente une données et un tableau contenant les classes correspondantes à chaque vecteurs.

```
import numpy as np

def split_train_test( data, annotation, ratio=0.2, seed=42 ):
    np.random.seed(seed)
```

```

liste = list(zip(data, annotation))
np.random.shuffle(liste)
data, annotation = zip(*liste)
size_test = int( len(data) * (1-ratio) )
return np.array(data[:size_test]), np.array(annotation[:size_test]),\
        np.array(data[size_test+1:]), np.array(annotation[size_test+1:]))

trainX, trainY, testX, testY = split_train_test( digits.data, digits.target)

print(trainY)

print("Taille du jeu du jeu d'apprentissage {} et taille du jeux de test {}".format(trainX.shape, testX.shape))

# exemple du caractère à la position 24
idx = 0
print("Caractère à la position {} dans le jeu de données, il est annoté ({}).format( idx, trainY[idx]))
plt.imshow(trainX[idx].reshape(8,8), cmap='gray_r')
plt.show()

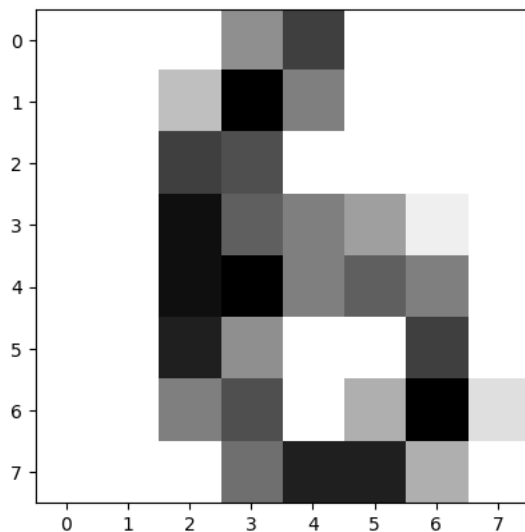
```

```

[6 9 3 ... 0 7 3]
Taille du jeu du jeu d'apprentissage (1437, 64) et taille du jeux de test (359, 64)

```

Caractère à la position 0 dans le jeu de données, il est annoté (6)



3. Préparation des données

Les images sont en nuance de gris sur 16 valeurs, les neurones de notre modèle sont de type on/off, on va donc devoir convertir nos jeux

Pour les annotations nous avons 10 classes la classe 0, classe 1 ... classe 9. Pour la même raison que les inputs on va devoir choisir une classe et redéfinir l'annotation à True pour la classe choisi et à False pour toutes les autres.

```

def modif_input( dataset, seuil = 4 ):
    for n in range(len(dataset)):
        for i in range(len(dataset[n])):
            dataset[n][i] = 1 if dataset[n][i] > seuil else 0
    return dataset

trainX = modif_input(trainX)
testX = modif_input(testX)

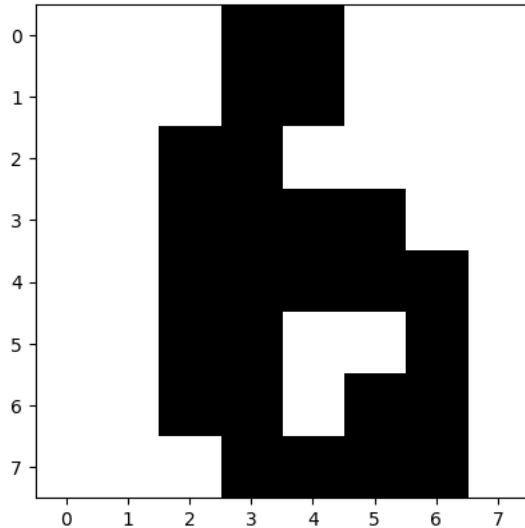
def modif_output( dataset, classe = 6 ):
    dataset = np.array(list(map( lambda x: x==classe, dataset )))
    return dataset

trainY = modif_output(trainY)
testY = modif_output(testY)

# exemple du caractère à la position 24
idx = 0
print("Caractère à la position {} dans le jeu de données, il est annoté ({}).format( idx, trainY[idx]))
plt.imshow(trainX[idx].reshape(8,8), cmap='gray_r')
plt.show()

```


Caractère à la position 0 dans le jeu de données, il est annoté (True)



4. Modèle d'apprentissage basé sur HEBB

Fonction fit pour l'apprentissage par renforcement des poids, apprentissage binaire, appartient à une classe ou n'appartient pas, retour le tableau des poids du model.

Fonction predict pour classer un tableau de vecteur donné.

```
import numpy as np

# apprentissage
def fit( X, Y ):
    weight = np.array([], float)
    # initialisation des poids
    for i in range(len(X[0])):
        weight = np.append( weight, 1 )

    # pour chaque vecteur du jeu de données
    for n in range(len(X)):
        for i in range(len(X[n])):
            weight[i] = weight[i] + X[n][i]*Y[n]

    return weight

# classification
def predict( X, weight, seuil = 2000 ):
    Y = []
    for n in range(len(X)):
        w = 0
        for i in range(len(X[n])):
            w = w + X[n][i]*weight[i]
        Y.append( w > seuil )
    return Y
```

5. Mesure de performance

Fonction calculant la précision et le rappel Fonction calculant la moyenne harmonique de la précision et du rappel

```
from collections import Counter

def performance( test, predic ):
    couple = list(zip(test,predic))
    nbrCouple = Counter(couple)
    TruePositif = nbrCouple[(True,True)]
    FalsePositif = nbrCouple[(False,True)]
    TrueNegatif = nbrCouple[(False,False)]
    FalseNegatif = nbrCouple[(True,False)]

    total_elements = TruePositif + FalsePositif + TrueNegatif + FalseNegatif

    preci = (TruePositif/(TruePositif+FalsePositif)) if ((TruePositif+FalsePositif)>0) else 0
    rappel = (TruePositif/(TruePositif+FalseNegatif)) if ((TruePositif+FalseNegatif)>0) else 0
    accuracy = (TruePositif + TrueNegatif) / total_elements if total_elements > 0 else 0
```

```
# Calcul du F1-score (moyenne harmonique de précision et rappel)
f1_score = (2 * preci * rappel) / (preci + rappel) if (preci + rappel) > 0 else 0

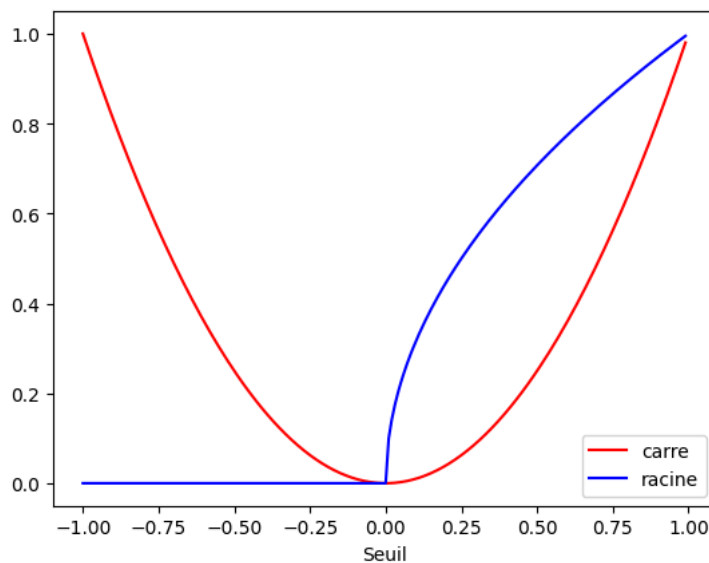
return preci, rappel, accuracy, f1_score
```

6. Exemple courbe

Code pour tracer un courbe

```
abscisse = [ i/100 for i in range(-100,100)]
courbe_un = []
courbe_deux = []
for x in abscisse:
    courbe_un.append( x**2 )
    courbe_deux.append( x**0.5 )

plt.plot(abscisse, courbe_un,"r-", label="carre")
plt.plot(abscisse, courbe_deux, "b-", label="racine")
plt.legend()
plt.xlabel("Seuil")
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

weights = fit(trainX, trainY)

seuils = np.arange(2200, 3400, 50)

sensibilites = []
precisions = []
accuracies = []
f1_scores = []
scores_f1 = []
scores_moyens = []

# Pour chaque seuil
for seuil in seuils:
    predictions = predict(testX, weights, seuil)
    precision, rappel, accuracy, f1_score_val = performance(testY, predictions)
    precisions.append(precision)
    sensibilites.append(rappel)
    f1_scores.append(f1_score_val)

    # Score F1 : moyenne harmonique
    if precision + rappel > 0:
        f1 = 2 * (precision * rappel) / (precision + rappel)
    else:
        f1 = 0
    scores_f1.append(f1)

    # Score moyen : moyenne arithmétique
    scores_moyens.append((precision + rappel) / 2)

plt.figure(figsize=(12, 7))
```

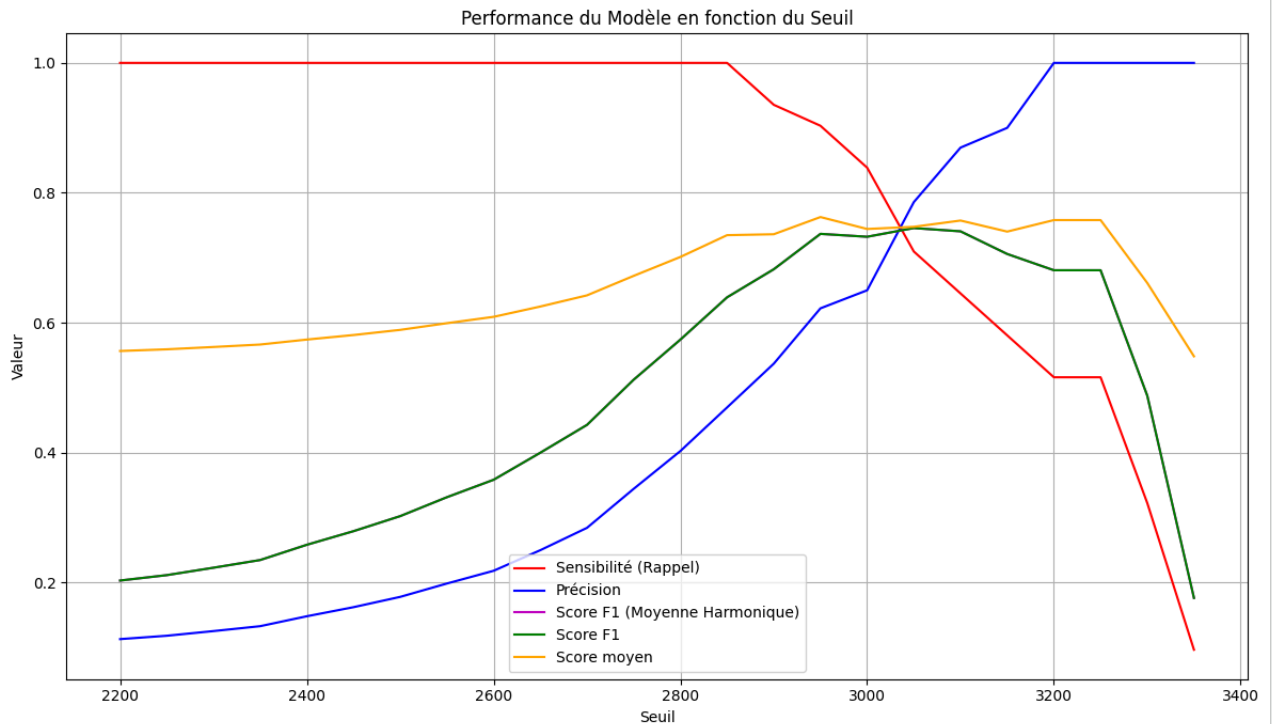
```

plt.plot(seuils, sensibilites, 'r-', label='Sensibilité (Rappel)')
plt.plot(seuils, precisions, 'b-', label='Précision')
plt.plot(seuils, f1_scores, 'm-', label='Score F1 (Moyenne Harmonique)')
plt.plot(seuils, scores_f1, 'g-', label='Score F1')
plt.plot(seuils, scores_moyens, 'orange', label='Score moyen')

plt.xlabel('Seuil')
plt.ylabel('Valeur')
plt.title('Performance du Modèle en fonction du Seuil')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

```



8. Courbe ROC

```

import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# Recalculate metrics for class 6 to include FPR
weights_class6 = fit(trainX, trainY)

seuils_roc = np.arange(2200, 3400, 10)

tpr_class6 = []
fpr_class6 = []

for seuil in seuils_roc:
    predictions_class6 = predict(testX, weights_class6, seuil)

    # Calculate TruePositif, FalsePositif, TrueNegatif, FalseNegatif
    couple = list(zip(testY, predictions_class6))
    nbrCouple = Counter(couple)

    TruePositif = nbrCouple[(True, True)]
    FalsePositif = nbrCouple[(False, True)]
    TrueNegatif = nbrCouple[(False, False)]
    FalseNegatif = nbrCouple[(True, False)]

    # True Positive Rate (rappel/sensitivity)
    tpr = (TruePositif / (TruePositif + FalseNegatif)) if ((TruePositif + FalseNegatif) > 0) else 0
    # False Positive Rate

```

```

fpr = (FalsePositif / (FalsePositif + TrueNegatif)) if ((FalsePositif + TrueNegatif) > 0) else 0

tpr_class6.append(tpr)
fpr_class6.append(fpr)

# Plot ROC curve for class 6
plt.figure(figsize=(8, 6))
plt.plot(fpr_class6, tpr_class6, 'b-', label='Courbe ROC (Classe 6)')
plt.xlabel('Taux de Faux Positifs (FPR)')
plt.ylabel('Taux de Vrais Positifs (TPR - Rappel)')
plt.title('Courbe ROC pour la Classe 6')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```

